

Chapter 5

Classes, objects and constructors

Object-oriented TypeScript, test first

Student Cards

6 students

Ada Lovelace

101 · Computing

Hello, Countess!

Alan Turing

102 · Mathematics

Hello, Alan!

Grace Hopper

103 · Computing

Hello, Amazing Grace!

Tim Berners-Lee

104 · Physics

Hello, Tim!

Margaret Hamilton

105 · Undecided

Hello, Margaret!

Linus Torvalds

106 · Computing

Hello, Linus!

Today

- classes and objects - what changes from Java
- constructors, and **parameter properties**
- default and optional parameters instead of overloading
- `toString()` - and where it is called for you
- JSON data -> objects with `map` + `new`
- what to test about a constructor
- a die that can be tested: pass the random number in

The words (as in Java)

- **class** - a blueprint: fields + methods
- **object** (instance) - one thing made from it, with its own values
- **constructor** - runs when `new` makes an object
- you know already: `export class` , types after names, `this.` always

Test first: the first test

```
Deno.test("a new student has the full name it was given", () => {  
  const student = new Student(101, "Ada", "Lovelace", "Computing");  
  
  assertEquals(student.getFullName(), "Ada Lovelace");  
});
```

```
error: Module not found "src/Student.ts".  
Tests: the tests could not run (see above)
```

The long way (Java style)

```
export class Student {  
  private id: number;  
  private firstName: string;  
  // ... surname, course  
  
  constructor(id: number, firstName: string /* ... */) {  
    this.id = id;  
    this.firstName = firstName;  
    // ...  
  }  
}
```

- always called `constructor` ; no return type
- `this.` is never optional - forget it and TypeScript suggests it

Parameter properties

The same class, written two ways

The long way (as in Java)

```
export class Student {  
  private id: number;  
  private firstName: string;  
  constructor(id: number,  
              firstName: string) {  
    this.id = id;  
    this.firstName = firstName;  
  }  
}
```

fields + parameters + assignments: 3 places

Parameter properties

```
export class Student {  
  constructor(  
    private id: number,  
    private firstName: string,  
  ) {}  
}
```

the word **private** in front of a parameter declares the field **and** sets it from the argument: 1 place, nothing to forget

■ the field

■ the parameter

■ copying one into the other

Refactor under a green test

```
export class Student {  
  constructor(  
    private id: number,  
    private firstName: string,  
    private surname: string,  
    private course: string,  
  ) {}  
}
```

- `private` in front: declares the field **and** sets it
- no modifier = just a parameter:

Property 'name' does not exist on type 'Student'.

Exercise 5.1 - Shorten Team

```
export class Team {  
  private score: number = 0;  
  
  constructor(private name: string) {}  
  // ...  
}
```

- `score` stays an ordinary field - it does not come from an argument
- the tests pass unchanged: a refactor

A default parameter - red first

```
const student = new Student(102, "Alan", "Turing");  
assertEquals(student.getCourse(), "Undecided");
```

TS2554 [ERROR]: Expected 4 arguments, but got 3.

```
-   undefined  
+   "Undecided"
```

- a missing argument is `undefined` at run time

The fix

```
const DEFAULT_COURSE = "Undecided";

export class Student {
  constructor(
    // ... id, firstName, surname
    private course: string = DEFAULT_COURSE,
  ) {}
}
```

- used when the argument is missing or undefined
- works in any function or method

Optional parameters

```
private nickname?: string,
```

```
public getGreetingName(): string {  
    if (this.nickname === undefined) {  
        return this.firstName;  
    }  
    return this.nickname;  
}
```

- type is `string | undefined` - TypeScript makes you check

Passing undefined

```
new Student(105, "Margaret", "Hamilton", undefined, "Maggie");
```

- arguments match parameters **by position**
- `undefined` triggers the default: course is "Undecided"
- fine for one or two; ugly for many (Challenge 6)

One constructor, not several

TS2392 [ERROR]: Multiple constructor implementations are not allowed.

Java	TypeScript
4-argument constructor	<code>new Student(101, "Ada", "Lovelace", "Computing")</code>
3 arguments + <code>this(...)</code>	<code>new Student(102, "Alan", "Turing")</code>

- types vanish at run time: nothing to choose by

Required first

```
constructor(a: string, b?: string, c: string) {}
```

A required parameter cannot follow an optional parameter.

- required, then defaults / ?
- only the end of the list can be left out

toString, test first

```
assertEquals(student.toString(),  
    "(Student) 101 Ada Lovelace, Computing");
```

```
- [object Object]  
+ (Student) 101 Ada Lovelace, Computing
```

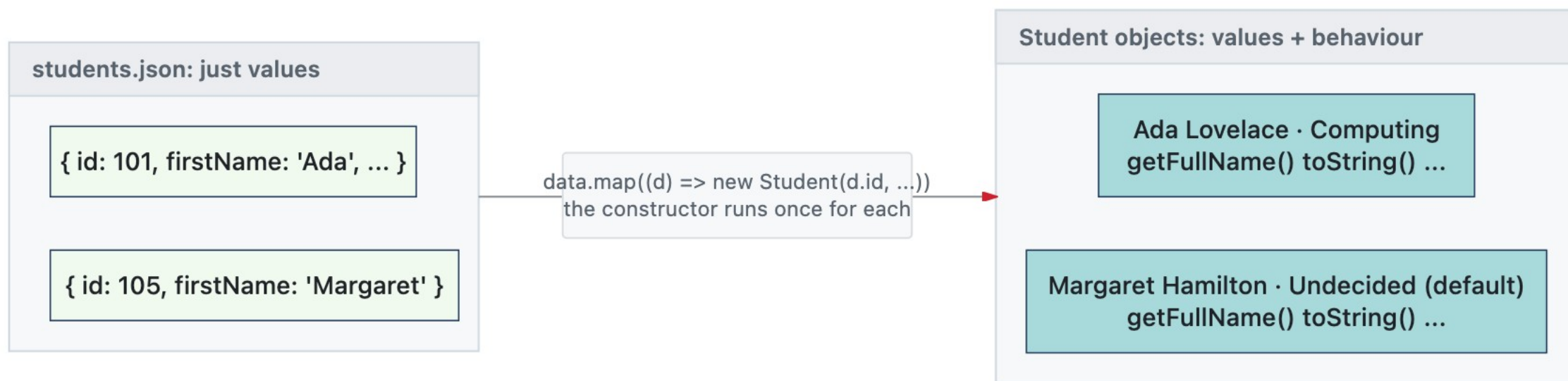
- **no type error**: every object already has a `toString()`
- returns a string - it does not print

Where toString is called for you

```
`${student}`           // "(Student) 102 Alan Turing, Undecided"  
String(student)        // the same  
modules.join("; ")     // each element's toString  
console.log(student)    // NOT: shows the fields
```

- no `override`: `Student` extends nothing (Chapter 11)

From plain data to objects



One class (the blueprint), as many objects as there are rows of data

studentsFrom

```
export type StudentData = {  
  id: number;  
  firstName: string;  
  surname: string;  
  course?: string;  
  nickname?: string;  
};  
  
export const studentsFrom = (data: StudentData[]): Student[] =>  
  data.map((d) => new Student(d.id, d.firstName, d.surname,  
    d.course, d.nickname));
```

- missing property -> undefined -> default. No if

Testing constructors

- test **your** decisions: values kept, defaults, optional both ways, `toString()`
- the compiler catches the rest:

Mistake	TypeScript says
<code>new Student(101)</code>	Expected 3–5 arguments, but got 1.
<code>Student(101, ...)</code>	... not callable. Did you mean to include 'new'?

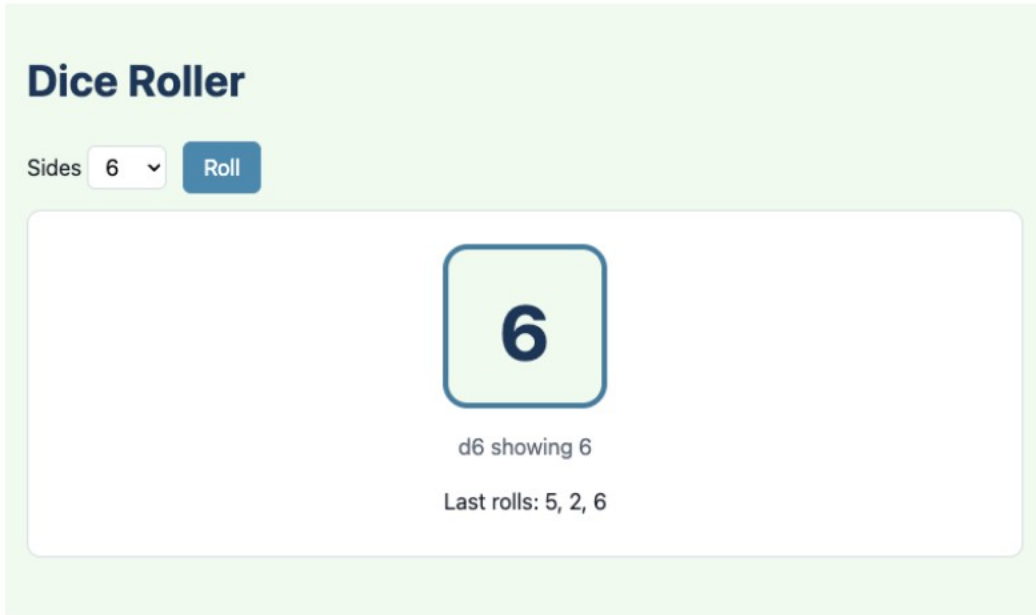
Two objects, same values

```
const a = new Student(1, "Ada", "Lovelace");  
const b = new Student(1, "Ada", "Lovelace");  
assertEquals(a, b);           // passes: same contents  
assertStrictEquals(a, b);     // fails: two objects
```

Values have the same structure but are not reference-equal.

- `assertEquals` also checks the class: `Student {` vs `{`

Project 2: a die



- `constructor(private sides: number = DEFAULT_SIDES) {}`
- `roll(random)` is **given** its random number

Rolling in small steps

```
assertEquals(die.roll(0), 1);
```

TypeError: die.roll is not a function

```
public roll(random: number): number {  
    return 1; // fake it  
}
```

- green - and the linter: ``random`` is never used

The next test forces the formula

```
assertEquals(die.roll(ALMOST_ONE), 6);    // - 1 + 6
```

```
public roll(random: number): number {  
    this.value = Math.floor(random * this.sides) + 1;  
    return this.value;  
}
```

- 0 -> 1, just below 1 -> 6, 0.5 -> 4, a d20 can roll 20

New objects while the page runs

```
let die = new Die();

sidesChoice?.addEventListener("change", () => {
  die = new Die(Number(sidesChoice.value));
  history = [];
  render();
});
```

- `Math.random()` is called only in `main.ts`

Project 3: undefined with a meaning

```
constructor(  
  private code: string,  
  private title: string,  
  private credits: number = DEFAULT_CREDITS,  
  private semester?: number,    // undefined: all year  
) {}  
  
public isYearLong(): boolean {  
  return this.semester === undefined;  
}
```

Class or type?

- **type** : the shape of plain data (JSON) - no constructor, no methods
- **class**: behaviour - defaults in one place, methods, private fields
- `StudentData` (shape) -> `studentsFrom` -> `Student` (object)

Java and TypeScript

Java	TypeScript
<code>this.id = id;</code> in the body	<code>constructor(private id: number) {}</code>
overloaded constructors	defaults <code>=</code> and optional <code>?</code>
<code>@Override toString()</code>	<code>toString(): string</code>
<code>Student@6d06d69c</code>	<code>[object Object]</code>
<code>a == b</code>	<code>assertStrictEquals</code> / <code>===</code>

Summary

- `constructor`, one per class; `this.` always
- parameter properties when the constructor just stores
- defaults and `?` replace overloading; required first
- `undefined` triggers a default
- `toString()` returns; template literals and `join` call it
- `data.map((d) => new Student(...))` - data becomes objects
- pass randomness in

Challenges

1. **Initials** - `getInitials()`
2. **Critical roll** - `isMaximum()`
3. **Who teaches it?** - optional `lecturer`
4. **Enough credits?** - `creditCheck(modules, target = 60)`
5. **A cup of dice** - `DiceCup`, a random **function**
6. **Too many arguments** - one object for the constructor